

INSTITUTO TECNOLÓGICO DE COSTA RICA  
Campus Tecnológico Central Cartago  
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras  
II Semestre 2024

Reporte #11 - Semana 12 – Sivarama - Capitulo 19  
Recursión

**Estudiante:** Mauricio González Prendas – **Carné:** 2024143009

**Profesor:** Esteban Arias Méndez

**Fecha de entrega:** 14/10/2024

**Bibliografía:**

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 391–401.

**Abstract**—*Recursion is a programming technique where a function calls itself, either directly or indirectly, to solve problems. This approach can elegantly express solutions like factorials or Fibonacci sequences. Recursion requires a termination condition to avoid infinite loops, and each call creates an activation record on the stack. While recursion can simplify code for problems that are naturally recursive, it also has downsides, such as inefficiency and increased memory usage compared to iteration. Examples of recursion include computing factorials, Fibonacci numbers, and implementing the Quicksort algorithm.*

---

## I. Recursión

La recursión es una técnica donde una función se llama a sí misma para resolver un problema. Existen dos tipos principales:

- Recursión directa: la función se llama a sí misma directamente.
- Recursión indirecta: una función llama a otra, que eventualmente llama a la primera.

La recursión es útil para expresar soluciones de forma elegante, como el cálculo de factoriales. Sin embargo, toda función recursiva debe tener una condición de terminación, que evita que la recursión continúe indefinidamente. El uso excesivo de la recursión puede llevar a problemas de eficiencia y mayor uso de memoria.

## II. Cálculo recursivo de factoriales

Un ejemplo clásico de recursión es el cálculo del factorial de un número, que puede expresarse como:

$\text{Factorial}(0) = 1$

$\text{Factorial}(n) = n * \text{Factorial}(n - 1)$ , para  $n > 0$

El siguiente código en C ilustra cómo implementar esta función recursiva:

```
int fact(int n) {
    if (n == 0) return 1;
    return n * fact(n - 1);
}
```

Cada llamada a la función factorial crea un registro de activación en la pila, que contiene los argumentos, la dirección de retorno y variables locales. Cuando la función termina, el registro es eliminado de la pila. La profundidad de la recursión determina el número de registros en la pila y, por tanto, el espacio en memoria utilizado.

### III. Ejemplo en ensamblador: cálculo recursivo de factoriales

El siguiente código en ensamblador calcula el factorial de un número de manera recursiva:

```
fact:
    cmp BL, 1
    jg one_up
    mov AX, 1
    ret
one_up:
    dec BL
    call fact
    inc BL
    mul BL
    ret
```

Este código recibe un número en el registro BL y devuelve el factorial en el registro AX. Si el número es mayor que 1, la función decrementa BL y llama nuevamente a la función con BL - 1. Al retornar de la recursión, incrementa BL y multiplica su valor por el resultado anterior. La condición de terminación ocurre cuando BL es menor o igual a 1.

### IV. Ejemplo: cálculo recursivo del número Fibonacci

El número Fibonacci  $n$  se define como:

$\text{Fib}(1) = 1$

$\text{Fib}(2) = 1$

$\text{Fib}(n) = \text{Fib}(n - 1) + \text{Fib}(n - 2)$  para  $n > 2$

En ensamblador, la implementación sería similar a la del factorial, llamando recursivamente a la función hasta alcanzar los casos base ( $n=1$  o  $n=2$ ).

### V. Ejemplo: algoritmo Quicksort recursivo

Quicksort es un algoritmo de ordenamiento que usa una estrategia de dividir y conquistar. Se selecciona un elemento pivote y se divide el arreglo en dos subarreglos: uno con elementos menores que el pivote y otro con elementos mayores. Luego, se aplica Quicksort de manera recursiva en cada subarreglo.

El siguiente código ilustra el funcionamiento básico de Quicksort:

```
void quick_sort(int array[], int lo, int hi) {
    if (hi > lo) {
        int x = array[hi];
        int i = lo;
        int j = hi;
        while (i < j) {
            while (array[i] < x) i++;
            while (array[j] > x) j--;
            if (i < j) swap(array[i], array[j]);
        }
        swap(array[i], array[hi]);
        quick_sort(array, lo, i - 1);
        quick_sort(array, i + 1, hi);
    }
}
```

Este algoritmo es eficiente en la mayoría de los casos, pero su rendimiento depende de la elección del pivote.

## VI. Comparación entre recursión e iteración

Aunque todo problema recursivo tiene una solución iterativa equivalente, la recursión es más elegante y refleja mejor la definición matemática en algunos casos. Sin embargo, la recursión tiene desventajas:

- Ineficiencia: la recursión añade sobrecarga al crear múltiples llamadas a funciones, lo que puede causar duplicación de cálculos.
- Mayor uso de memoria: cada llamada recursiva utiliza un registro de activación, lo que incrementa el uso de la pila, especialmente para problemas con gran profundidad recursiva.